Créer un mod pour Claude Code

Ce qu'un mod peut faire, où il agit à l'écran, et comment en écrire un, le tester et le partager.

1 Qu'est-ce qu'un mod ?

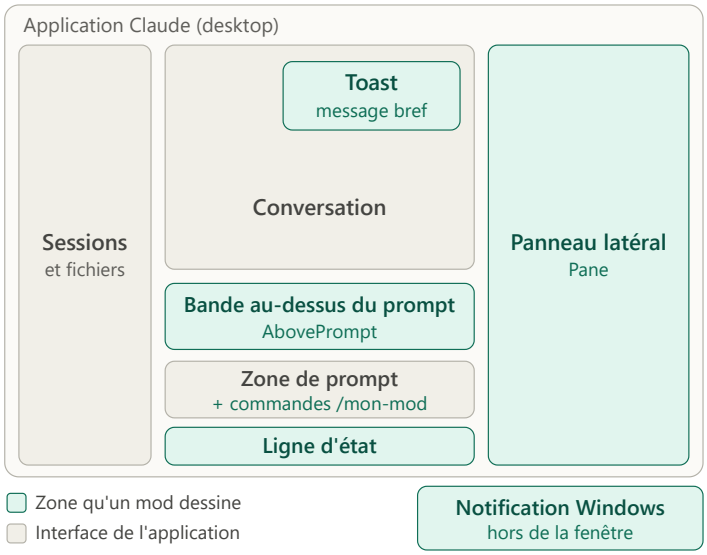
Un mod est un plugin Claude Code dont le code (TypeScript) se branche sur les **événements** du moteur : le rendu de l'interface, l'envoi d'un prompt, un appel d'outil, la fin d'un tour… Il tient en trois fichiers :

```
mon-mod/
├─ .claude-plugin/plugin.json   nom, version, réglages
└─ hooks/
   ├─ hooks.json                désigne register.tsx
   └─ register.tsx              le code
```

```
export const register: Register = (on, options) => {
  on('turn.complete', ($, e, next) => {
    $.ui.toast('Tour terminé')
    return next(e)   // laisse la suite se dérouler
  })
}
```

Chaque hook reçoit `$` (tout ce que le moteur sait faire : `$.ui`, `$.session`, `$.store` …), `e` (les données de l'événement) et `next`. Appeler `next(e)` laisse l'événement continuer ; `next({ ...e, text })` le modifie ; ne pas l'appeler le remplace. Le module n'a ni DOM ni Node : tout passe par `$`.

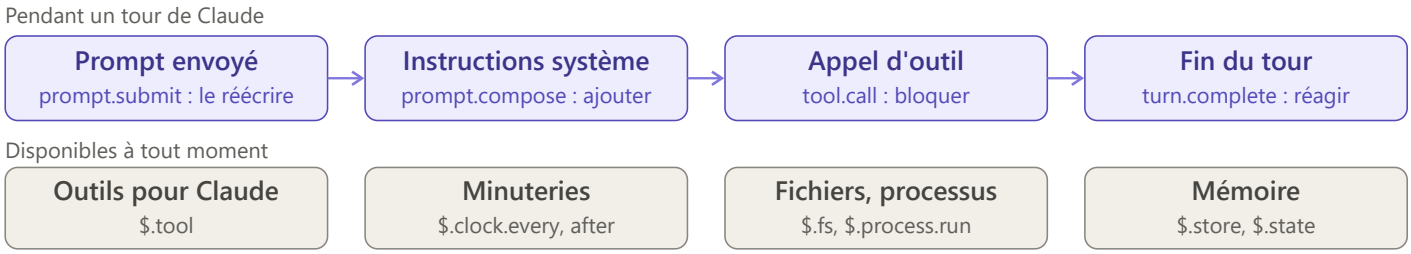
2 Où un mod agit à l'écran



Zone	Comment l'obtenir
Bande	hook `ui.render` sur `{ component: 'AbovePrompt' }` ; renvoie un arbre, et `next(e)` pour garder les autres bandes
Panneau	`$.ui.open({ id, title })` puis hook `ui.render` sur `{ component: 'Pane', requestId: id }`
Toast	`$.ui.toast(texte)` depuis n'importe quel hook
Notification	`$.ui.notify(texte, { title })`
Ligne d'état	`$.ui.status(texte)` ; `undefined` l'efface
Commande /	`$.command.register({ name })` dans `session.start`, réponse dans un hook `command.run`
Son	`$.audio.play({ asset })` ou `{ base64, mime }`

Les éléments viennent de la surface : `const { Box, Text, Svg } = $.ui.resolve(e)`. `e.surface` vaut `terminal`, `desktop`, `vscode` ou `mobile`. Sur le desktop, un `Svg` permet une carte graphique ; dans le terminal, seulement du texte.

3 Ce qu'un mod fait sans rien afficher



- **Prompt envoyé** : ajouter une consigne ou corriger le message avant que Claude le lise (`next({ ...e, text })`).
- **Instructions système** : ajouter une section que Claude lit à chaque tour.
- **Appel d'outil** : autoriser, bloquer (`return { deny: 'raison' }`), changer les paramètres, ou réagir au résultat. C'est la base d'un garde-fou.
- **Fin du tour** : toast, son, notification, ou mise à jour d'une bande (`$.ui.invalidate('ui.render')`).
- **Mesures** : `session.measure` signale chaque changement du contexte, des limites d'usage ou du coût ; `$.session.usage()` les lit.

4 Réglages et mémoire

- **Réglages** : déclarés sous `userConfig` dans le `plugin.json`, reçus dans `options`. Un champ `string` avec `options: ["auto", "sombre", "clair"]` devient une liste au choix dans `/config` ; les valeurs vivent dans `settings.json` sous `pluginConfigs["mon-mod@depot"].options`.
- **Mémoire** : `$.store.get/set` garde des données entre les sessions ; `$.state` garde des valeurs pour la session et redessine ce qui les lit (à déclarer dans `types/index.d.ts`). Les variables du module repartent de zéro à chaque rechargement.

5 Créer un mod pas à pas

1. **Demander à Claude.** Le plus simple : décrire le mod voulu dans une session (« une bande qui affiche… », « un panneau qui… »). Claude écrit les fichiers et, après votre accord (« Turn on the mods »), charge le mod à la fin du tour.
2. **Écrire les trois fichiers** à la main sinon. Le `plugin.json` contient au minimum `name`, `version` et `description`.
3. **Vérifier** :

   ```
   claude plugin validate mon-mod     # ce que le mod branche et ce que le moteur refuserait
   claude plugin test mon-mod         # exécute les fichiers *.test.ts(x) du mod
   ```

   Un test monte la surface voulue (`$.ui.mount({ ..., surface: 'terminal' })`) et cherche ce qu'elle affiche (`ui.find({ type: 'Text', text: /…/ })`).
4. **Charger pendant le développement** : `claude --plugin-dir mon-mod` pour une session, ou la variable `CLAUDE_CODE_PLUGIN_DIRS` dans le bloc `env` de `~/.claude/settings.json` (chemins séparés par `;` sous Windows). Une modification du dossier est rechargée sans réinstaller.
5. **Partager** : un dépôt GitHub avec `.claude-plugin/marketplace.json` qui liste les mods, puis `claude plugin install mon-mod --marketplace compte/depot`. Sur le desktop : **+** à côté du prompt › Plugins › Add marketplace › Add plugin.
6. **Mettre à jour** : augmenter `version` dans le `plugin.json` (sinon la mise à jour ne voit rien), pousser, puis `claude plugin marketplace update depot` et `claude plugin update mon-mod@depot`, et redémarrer l'application.

6 Exemple : une bande au-dessus du prompt

```
import type { Register } from 'claude-code'
export const register: Register = on => {
  // Redessiner la bande quand une mesure change (après chaque réponse).
  on('session.measure', ($, e, next) => { $.ui.invalidate('ui.render'); return next(e) })
  on('ui.render', { component: 'AbovePrompt' }, async ($, e, next) => {
    const { context } = await $.session.usage()
    const { Box, Text } = $.ui.resolve(e)
    const below = await next(e)              // les bandes des autres mods
    return (
      <Box flexDirection="column">
        {below}
        <Text color="green">Contexte : {context.percent ?? '—'} %</Text>
      </Box>
    )
  })
}
```

7 Pièges rencontrés et bonnes pratiques

- **Terminal** : la table d'éléments du terminal contient aussi `Svg` mais ne le dessine pas. Tester `e.surface !== 'terminal'`, et pas `'Svg' in table`, sinon le terminal n'affiche rien.
- **Cartes SVG** : elles sont affichées comme des images. Le texte ne revient pas à la ligne seul (découper en `<tspan>`) ; préfixer les `id` des dégradés pour ne pas mélanger deux cartes ; passer les couleurs par des variables CSS et `@media (prefers-color-scheme: light)` pour suivre le thème de l'application.
- **Largeur d'un panneau** : `e.props.bodyColumns` la donne en colonnes de texte (environ 7 px chacune).
- **Mesures** : `$.session.usage()` est vide avant la première réponse d'une session ; enregistrer la dernière mesure dans `$.store` et la reprendre au démarrage. Ce stockage dépend du mode de chargement : passer du dossier au marketplace repart de zéro.
- **Son sous Windows** : `$.audio.play` n'y trouve pas de lecteur. Jouer un WAV avec PowerShell (`$.process.run`, `System.Media.SoundPlayer`).
- **Installation** : ne pas combiner `CLAUDE_CODE_PLUGIN_DIRS` et le marketplace pour un même mod (il s'exécuterait deux fois). Les mods s'installent machine par machine : une session SSH lit le `~/.claude` du serveur.
- **Comprendre une erreur** : un hook qui échoue est ignoré seul. Pour un mod chargé depuis son dossier, une ligne grise dans la conversation dit pourquoi ; sinon, le journal `claude --debug`.

> **Référence complète** : l'API (environ 14 000 lignes de types commentés, avec un exemple par fonction) est écrite à côté de chaque mod chargé, dans `.claude-plugin/types/claude-code/index.d.ts`. Chercher le nom d'un événement (`'tool.call'`) ou d'un élément (`Pane`) y donne ses entrées, ses résultats et un exemple. L'API est en accès anticipé et peut évoluer d'une version à l'autre.

Guide rédigé à partir des mods context-weather, usage-limits, usage-forecast et turn-alert (dépôt Teddy57600/teddy-mods), avec Claude Code 2.1.309. Novembre 2026.